



LLM Fine-Tuning

A Comprehensive Practitioner's Guide

Finetuning · Tools & Platforms · Hyperparameters · Real-Time Use
Cases

Deep Learning & NLP Series

2025 Edition

CHAPTER 1

Fine-Tuning: Fundamentals

What Is Fine-Tuning?

Fine-tuning is the process of taking a pre-trained large language model (LLM) — one already trained on vast amounts of general text — and continuing its training on a smaller, task-specific dataset. The model's weights are updated so that it specialises in the target domain, tone, or task format, without forgetting all general knowledge it already possesses.

■ *Key insight: Fine-tuning does NOT teach a model facts from scratch. It reshapes how the model applies existing knowledge toward a specific goal.*

Pre-training vs Fine-Tuning vs Prompting

Aspect	Pre-training	Fine-Tuning	Prompting
Compute cost	Massive (GPU-months)	Moderate	Negligible
Data needed	Trillions of tokens	Hundreds–millions	Zero (few-shot)
Weight update	Yes – from scratch	Yes – from base	No
Customisation	Architecture only	Deep task-specific	Surface-level
Inference cost	N/A	Same as base	Same as base
Skill required	Research team	ML engineer	Prompt engineer

Why Fine-Tune?

- **Domain adaptation** — Legal, medical, finance, or code tasks need specialised vocabulary and reasoning patterns.
- **Consistent format** — Force output to always be JSON, YAML, SQL, or a specific template.
- **Tone & persona** — Make a customer-support bot always polite, concise, and on-brand.
- **Reduced prompt length** — Encode instructions into weights, slashing token costs at inference.
- **Privacy** — Fine-tune on-premise so proprietary data never leaves your infrastructure.
- **Better accuracy** — Outperform even very long prompts for narrow, well-defined tasks.

Types of Fine-Tuning

Full Fine-Tuning

All model parameters are updated during training. Achieves the best task performance but requires large GPU memory (often multi-GPU setups) and risks *catastrophic forgetting* — where the model loses general capabilities while specialising.

Parameter-Efficient Fine-Tuning (PEFT)

Only a small subset of parameters (or newly added adapter parameters) are trained. The backbone weights stay frozen. PEFT methods use <5 % of the compute of full fine-tuning while delivering 90–98 % of the performance gains.

Method	Mechanism	Performance	Efficiency
LoRA	Low-rank matrices injected into attention layers	★★★★★	★★★★■
QLoRA	LoRA + 4-bit quantised base model	★★★★★	★★★★★
Prefix	Soft token prefixes prepended to each layer	★★★★■	★★★★■
Adapters	Small bottleneck layers inserted between blocks	★★★★■	★★★★■
IA ³	Learned scaling vectors for keys, values, FF	★★★★■	★★★★★
Prompt Tuning	Tunable soft prompts, weights frozen	★★■	★★★★★

Instruction Fine-Tuning (IFT): Train on (instruction, response) pairs to make a base model follow natural-language directives. Used by GPT-4, LLaMA-3-Instruct, Mistral-Instruct, etc.

RLHF / RLAIF / DPO: Reinforcement Learning from Human (or AI) Feedback aligns model outputs to human preferences. Direct Preference Optimisation (DPO) achieves similar results without a separate reward model, making it faster and more stable.

CHAPTER 2

Tools & Platforms

The Fine-Tuning Stack

A complete fine-tuning pipeline involves libraries for model loading, training loops, parameter-efficient methods, and serving. Below is a curated map of the most widely used options at each layer.

Core Training Frameworks

Framework	Key Feature	Language	Best For
Hugging Face Transformers	Model hub + Trainer API	Python	All major LLMs
PyTorch Lightning	High-level training loop	Python	Research & prod
DeepSpeed	ZeRO sharding, gradient checkpointing	Python/C++	Multi-GPU/TPU
Megatron-LM	NVIDIA tensor parallelism	Python/CUDA	Very large models
Axolotl	Config-driven fine-tuning wrapper	YAML/Python	LLaMA, Mistral
LLaMA-Factory	GUI + CLI for many PEFT methods	Python	LLaMA family
Unsloth	2× speed, 70 % less VRAM vs vanilla	Python	LLaMA, Mistral, Gemma

PEFT Libraries

Hugging Face PEFT is the de-facto standard, offering LoRA, QLoRA, IA3, prefix tuning, prompt tuning, and adapters with a unified API. **bitsandbytes** provides GPU-accelerated 4-bit / 8-bit quantisation. **trl (Transformer Reinforcement Learning)** adds SFTTrainer, RewardTrainer, and DPOTrainer on top of PEFT.

Managed Cloud Platforms

Platform	Supported Models	Interface	Highlight
OpenAI Fine-Tuning API	GPT-3.5-Turbo, GPT-4o	JSONL upload → API	No GPU management, fast
Google Vertex AI	Gemini, PaLM 2	Managed pipeline	GCP integration
AWS SageMaker	Any (bring your own)	Studio + HF DLC	Enterprise, VPC
Azure AI Foundry	GPT-4, Phi-3, Llama	Studio GUI	Microsoft ecosystem
Together AI	Open models	REST API	Fast, cheap PEFT
Fireworks AI	Open models	REST API	Production inference
Anyscale	LLaMA, Mistral	Ray-based	Scalable clusters
Modal	Any open model	Python SDK	Serverless GPU
RunPod	Any open model	SSH / Jupyter	Cheap GPU rentals

Platform	Supported Models	Interface	Highlight
Lambda Labs	Any open model	SSH / API	A100/H100 on-demand

Data Preparation Tools

- **Argilla** — Open-source annotation & data labelling with a web UI.
- **Label Studio** — Multi-modal annotation (text, images, audio).
- **Datasets (HF)** — Efficient streaming, shuffling, and tokenisation for large corpora.
- **DVC** — Git-like version control for datasets and model artefacts.
- **LangSmith** — Trace, evaluate, and curate LLM outputs for training data.
- **RAGAS** — Automated RAG and fine-tuning evaluation metrics.

Monitoring & Experiment Tracking

- **Weights & Biases (W&B;)** — Loss curves, gradient norms, GPU utilisation, model artefacts.
- **MLflow** — Open-source experiment tracking, model registry.
- **TensorBoard** — Built into TF/PyTorch, basic loss and histogram visualisation.
- **Comet ML** — Cloud experiment tracking with LLM-specific panels.

CHAPTER 3

Hyperparameters

Hyperparameters are the knobs you tune before training begins. Getting them right dramatically affects convergence speed, final accuracy, and training stability. This chapter covers every major hyperparameter and its practical guidance.

Learning Rate (LR)

The single most impactful hyperparameter. Controls how much model weights shift each gradient step. Too high → divergence, NaN loss. Too low → painfully slow convergence.

Scenario	Typical Range	Notes
Full fine-tune (7B model)	1e-5 – 5e-5	5e-6 too slow; 1e-4 diverges
LoRA / QLoRA	1e-4 – 3e-4	More aggressive since adapters are tiny
Instruction tuning	2e-5 – 1e-4	Lower for alignment tasks
Continued pre-training	5e-6 – 2e-5	Conservative to preserve general knowledge

LR Schedulers

- **Cosine with warmup** — Most popular. LR warms from 0 → peak over first 3–10 % of steps, then decays as a cosine wave.
- **Linear decay** — Simpler; decays linearly after warmup. Good default for short runs.
- **Constant LR** — Use only with very small datasets or short experiments.
- **OneCycleLR** — Cycles through low → high → low; can find good solutions faster.

Batch Size & Gradient Accumulation

Effective batch size = per-device batch size × number of GPUs × gradient accumulation steps. Larger effective batches stabilise training but require more memory. Gradient accumulation simulates a large batch on limited VRAM by accumulating gradients over multiple forward passes before doing one weight update.

■ *Rule of thumb: Start with effective batch size 32–128. If you see very noisy loss, increase it. If GPU is OOM, halve per-device batch and double accumulation steps.*

Epochs & Steps

- **Epochs** — How many full passes over your dataset. 1–3 epochs is usually enough for SFT on curated data; more causes overfitting.
- **Max steps** — Explicit step limit; overrides epochs. Preferred when dataset size is uncertain.
- **Early stopping** — Monitor validation loss; stop when it stops improving (patience = 2–5 evals).

Sequence Length (max_seq_length)

Determines the maximum token length of each training sample. Longer sequences capture more context but cost quadratic memory in attention layers. Flash Attention 2 reduces this to linear memory, enabling longer

sequences on the same hardware.

Task	Typical Length	Reason
Chat / QA	512 – 2 048	Most turns fit easily
Summarisation	2 048 – 4 096	Document + summary
Code generation	2 048 – 8 192	Functions, classes, files
Long-document tasks	8 192 – 32 768	Requires Flash Attention 2

LoRA-Specific Hyperparameters

Parameter	Common Values	Guidance
r (rank)	8, 16, 32, 64	Higher r = more expressiveness, more parameters. Start at 16.
alpha (α)	$r \times 1$ or $r \times 2$	Scaling factor. α/r controls effective LR of LoRA weights.
dropout	0.05 – 0.1	Regularisation for LoRA layers. Use 0 if dataset is large.
target_modules	q_proj, v_proj...	Which layers to apply LoRA. Adding k_proj, o_proj helps.
bias	none / lora_only	Whether to train bias terms. 'none' is safest default.

Other Key Hyperparameters

Hyperparameter	Typical Value	Notes
Weight decay	0.01 – 0.1	L2 regularisation; prevents overfitting on small datasets
Warmup ratio	0.03 – 0.1	Fraction of steps used for LR warmup
Grad clipping	0.3 – 1.0	Clip gradient norm; prevents explosive gradients
Dropout	0 – 0.1	Disable (0) for very small datasets to maximise signal
Precision	bf16 / fp16	bf16 preferred on Ampere+; more numerically stable
Packing	True / False	Pack multiple short samples into one sequence to improve GP
NEFTune noise α	5 – 15	Adding noise to embeddings during training boosts instruction

CHAPTER 4

Real-Time Use Cases

Fine-tuned models are deployed across virtually every industry. Below are the most impactful production use cases, along with the specific fine-tuning strategy that works best for each.

1. Customer Support Automation

Fine-tuned LLMs handle tier-1 support tickets automatically — answering FAQs, classifying issues (billing, technical, returns), and drafting escalation summaries. They learn brand voice and product-specific terminology that generic models lack.

Strategy: Instruction fine-tuning (SFT) on (ticket, ideal-response) pairs. 5 000–50 000 pairs is enough. Add DPO to rank helpful vs unhelpful responses.

Data tip: Export resolved tickets from Zendesk/Freshdesk. Filter for 4–5 star CSAT. Remove PII.

2. Clinical NLP & Medical Documentation

Models fine-tuned on clinical notes extract diagnoses, medications, and dosages (Named Entity Recognition), summarise discharge letters, and auto-populate EHR fields. Outperform GPT-4 on MIMIC and i2b2 benchmarks by 15–30 %.

Strategy: QLoRA on domain-specific corpora (PubMed, MIMIC-III), followed by SFT on task-specific examples. Keep medical entity labels consistent (SNOMED CT, ICD-10).

Data tip: De-identify notes with Presidio. Use MIMIC-III or synthetic data (Synthea). 10K–200K clinical notes ideal.

3. Code Generation & Review

Enterprise code assistants fine-tuned on internal codebases, coding standards, and proprietary APIs. Generate boilerplate, write unit tests, explain legacy code, detect vulnerabilities, and enforce style guides. Tools like GitHub Copilot use this exact approach.

Strategy: Continued pre-training on internal code repos (GitHub/GitLab exports), then SFT on (instruction, code) pairs. Pack files into sequences. Use fill-in-the-middle (FIM) objective.

Data tip: Dump all internal repos. Filter: remove auto-generated files, binaries, secrets. ~1B tokens of internal code saturates smaller models.

4. Legal Document Analysis

Fine-tuned models review contracts to flag unusual clauses, extract key obligations and dates, classify risk levels, and summarise lengthy agreements. Law firms report 70–90 % time savings on contract review.

Strategy: SFT on annotated contracts (clause type labels, risk labels, summary pairs). Few thousand annotated documents suffice due to domain density. Use longer context (8K+).

Data tip: Annotate with legal experts. Use EDGAR filings for pre-training. Synthetic data via GPT-4 for augmentation on rare clause types.

5. Financial Analysis & Reporting

Sentiment analysis on earnings calls and news (FinBERT-style but more powerful), structured extraction from financial statements (revenue, EPS, guidance), and automatic research report generation from data tables.

Strategy: SFT on (financial text, structured output) pairs. Combine with RAG for live data. DPO for report quality ranking (analyst-approved vs rejected drafts).

Data tip: SEC EDGAR filings, earnings call transcripts. Label sentiment with analyst ratings. FinancialPhraseBank for sentiment benchmarking.

6. Personalised Education & Tutoring

Tutoring models that adapt explanations to student level, generate practice problems, give step-by-step worked solutions, and provide Socratic hints rather than direct answers. Deployed in apps like Khan Academy's Khanmigo.

Strategy: SFT on (student question, pedagogically correct answer) pairs. Reinforcement learning to reward teaching quality over answer-giving. Curriculum-specific fine-tuning per grade.

Data tip: Curate solved problem sets with expert explanations. Avoid 'just give the answer' examples. Tag by subject, grade, difficulty.

7. Content Moderation

Classifiers fine-tuned to detect hate speech, misinformation, NSFW content, spam, and policy violations. Deployed at inference in real-time pipelines. Much faster and cheaper than calling a general LLM for every piece of content.

Strategy: Classification fine-tuning on labelled violation examples. Binary or multi-label (hate speech, violence, spam, etc.). Distil a small BERT-class model for low latency.

Data tip: Human-labelled moderation queues. Use active learning: model labels → human review → retrain. Balance classes; violations are rare events.

8. Multilingual & Cross-lingual Tasks

Fine-tune multilingual base models (mT5, BLOOM, Aya) for low-resource languages, domain-specific translation (medical, legal), and cross-lingual information retrieval. Critical for markets underserved by English-centric models.

Strategy: Continued pre-training on target-language text, then SFT on bilingual pairs. Cross-lingual transfer: fine-tune in English, evaluate zero-shot in target language.

Data tip: CC-100, OPUS corpora for multilingual pre-training. Professional human translation for SFT pairs; MT + post-editing is cheaper for low-resource.

CHAPTER 5

Pipeline & Best Practices

End-to-End Fine-Tuning Pipeline

1. Define objective	What exact task? What metric? What's the baseline?
2. Curate dataset	Collect, clean, deduplicate, split (80/10/10). Min: 500 examples; ideal: 5K+.
3. Choose base model	Match size to compute budget. LLaMA-3 8B or Mistral-7B are strong defaults.
4. Select PEFT method	QLoRA is the default for GPU-constrained setups. Full FT for max accuracy.
5. Set hyperparameters	Use cosine LR schedule, warmup 5 %, bf16 precision, gradient clipping 1.0.
6. Train & monitor	Watch train/val loss gap. Log to W&B.; Save checkpoints every 500 steps.
7. Evaluate rigorously	Task-specific metrics (ROUGE, BLEU, pass@k, F1) + human evaluation.
8. Merge & quantise	Merge LoRA → base. Quantise to GGUF (llama.cpp) or AWQ for deployment.
9. Deploy & monitor	vLLM or TGI for serving. Monitor latency, error rate, and output drift.

Common Pitfalls & How to Avoid Them

Pitfall	Symptom	Fix
Catastrophic forgetting	Full fine-tuning replaces general skills	Use PEFT; add diverse general data to training mix (5-10%)
Overfitting	Val loss rises while train loss falls	Reduce epochs (1–2); add dropout; increase dataset size
Data leakage	Test examples in training set	Deduplicate across splits; use time-based splits for temporal tasks
Label noise	Inconsistent or wrong annotations	Audit 5 % of data manually; use inter-annotator agreement
Format mismatch	Model ignores instruction format	Apply the same chat template as the base model (e.g. LLaMA)
OOM crashes	GPU runs out of memory	Enable gradient checkpointing; reduce batch size; use 4-bit quantization

Quick-Reference Checklist

- Dataset deduplicated and split (train / val / test) ■ Chat template matches base model format ■ Validation loss monitored every epoch ■ Gradient checkpointing enabled ■ bf16 or fp16 mixed precision set ■ LoRA rank and alpha chosen (start: $r=16$, $\alpha=32$) ■ LR scheduler = cosine with 5 % warmup ■ W&B; / MLflow logging connected ■ Checkpoint saved every 500 steps ■ Evaluation on held-out set before deployment

LLM Fine-Tuning Guide · 2025 Edition
Compiled for ML practitioners and AI engineers.